Reporte de despliegue: API REST en contenedor - Equipo 11

TC5061 MLOps · Semana 8 · Repositorio: https://github.com/oscartrix/TC5061-Proyecto-Equipo11

Repo: https://github.com/oscartrix/TC5061-Proyecto-Equipo11

Ramas de trabajo: `feat/api-contenedor` (#34), `ci/api-contenedor` (#35), `feat/inferencia-onnx` (#36)

Fecha de la auditoría: 2026-11-06

1. Estado inicial

Área	Estado en `main`	Brechas
Carga desde el Model Registry	`mi_bici/api/model.py` (PR #32): alias `champion` o versión fija con `MODEL_NAME`, `MODEL_ALIAS`, `MODEL_VERSION`	Solo funciona con un servidor MLflow accesible
Esquemas Pydantic	`schemas.py` (PR #31): `strict`, rangos, `zone` acotada, `is_weekend` coherente con `day_of_week`, 1 a 500 filas	Ninguna
Endpoints	PR #33 abierto: `/health`, `/model-info`, `/p1/predict`	Sin revisión; sin fusionar
Contenedor	`Dockerfile` levanta el servidor MLflow, copia `.git/` y no ejecuta uvicorn	No existe imagen de la API
CI	`calidad` + `pipeline` (semana 7 y registro de modelos)	No construye ni prueba la imagen
Documentación	README con la API local	Sin instrucciones de contenedor
Evidencia	—	Sin capturas del contenedor

2. Revisión de PRs de compañeros

PR	Autor	Contenido	Decisión
#31 `feat/api-esquemas`	Chuy	Esquemas de entrada y salida	Ya fusionado; sin cambios
#32 `feat/api-modelo-registry`	Chuy	Carga del modelo por nombre y alias/versión, con firma obligatoria	Ya fusionado; sin cambios
#33 `feat/api-endpoints`	Chuy	Endpoints, manejo de 422 y 500 sin detalle interno, recorte a 0, pruebas y README	107 pruebas pasan en local y el CI estaba en verde. Aprobado y fusionado a `main`

Las demás ramas remotas (`docs/*`, `feat/data-*`, etc.) ya estaban integradas; no se tocaron.

3. Remediación realizada

Área	Cambio
Modelo	`scripts/export_model.py` descarga el champion de P1 a `models/p1_champion/` (ignorado por git) y guarda nombre, versión y corrida en `registry.txt`
API	`load_served_model` usa `MODEL_PATH` si existe; si no, el registry como antes. Prueba nueva de carga local
Contenedor	`Containerfile` con `python:3.13-slim`, `requirements-api.txt` (solo lo que usa la API), usuario sin privilegios y `HEALTHCHECK`. Se conserva el `Dockerfile` del servidor MLflow

Área	Cambio
Makefile	`api-model`, `api-build`, `api-run`; `CONTAINER_ENGINE` cambia entre docker y podman
CI	El job `pipeline` exporta el champion, construye la imagen, levanta el contenedor, prueba 200 y 422 y guarda la respuesta
README	Sección de contenedor con comandos para Docker y Podman, variables, petición y respuesta, y la estrategia de despliegue
Extra	ONNX Runtime opcional con `INFERENCE_BACKEND=onnx` (PR #36 independiente, ver `analisis_onnx.md`)

4. Resultados

• Pruebas: 335 pasan (`evidencia/01_pytest.txt`).

• Imagen `mibici-api`: 814 MB; contenedor `healthy` en unos 10 s sin servidor MLflow (`02_build.txt`, `03_contenedor.txt`).

• Petición válida: `200` con `{"model_name":"mibici-equipo11-p1-demanda","model_version":"1","predictions":[10.182341634926482]}`; petición con `month: 13` y campos faltantes: `422` (`04_peticion.txt`).

• Reproducibilidad: la imagen se reconstruyó con `--no-cache` y dio la misma predicción.

• Champion: HGB v1, entrenado en esta máquina con los datos de `data/processed/`; los valores pueden diferir en el registry de otro integrante.

• ONNX: ver `analisis_onnx.md` (latencia 7 a 46 veces menor; diferencia media 0.088 viajes con el HGB).

5. Hallazgos

• El `Dockerfile` existente copia `.git/` solo para registrar el commit; la imagen de la API no lo necesita y por eso es un archivo aparte.

• La imagen inicial pesaba 1.16 GB; al quitar matplotlib, pillow, skops y pyarrow bajó a 814 MB.

• `mlflow` completo no es necesario para servir: basta `mlflow-skinny` y `psutil`.

• Los PRs #35 y #36 apuntan a ramas y no a `main`, así que el workflow (`on: pull_request` hacia `main`) no se dispara solo; el CI de #35 se lanzó con `workflow_dispatch`.

• Podman no está instalado en la máquina de la auditoría; el `Containerfile` se probó con Docker y los comandos de Podman son equivalentes, sin ejecutar.

• ONNX con el HGB cambia la predicción en parte de las filas por usar float32 (detalle en el análisis).

6. Aportes por integrante (commits en `main` desde 2026-11-01)

Integrante	Commits
Chuy	10
Jimena	2
Luis	1
Oscar	1 (más las ramas de esta entrega)

7. Pendiente (requiere al equipo)

• Que Jimena o Luis revisen y aprueben #34; la regla de `main` pide una aprobación distinta del autor.

• Cuando se fusione #34, cambiar la base de #35 y #36 a `main` y verificar el workflow completo.

• Que Jimena o Luis prueben el `Containerfile` con Podman y anoten el resultado en el README.

• Dar a Jimena y Luis una tarea de la entrega (por ejemplo, ampliar pruebas de la API o la prueba en Podman) para equilibrar los commits.

• Tomar la captura de la API desde su propio equipo si se prefiere sobre las generadas en `capturas/`.

• Decidir si se adopta ONNX; antes, definir la tolerancia de diferencia para el HGB.

Evidencia

Pruebas

```
    warnings.warn(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
335 passed, 19 warnings in 100.91s (0:01:40)
```

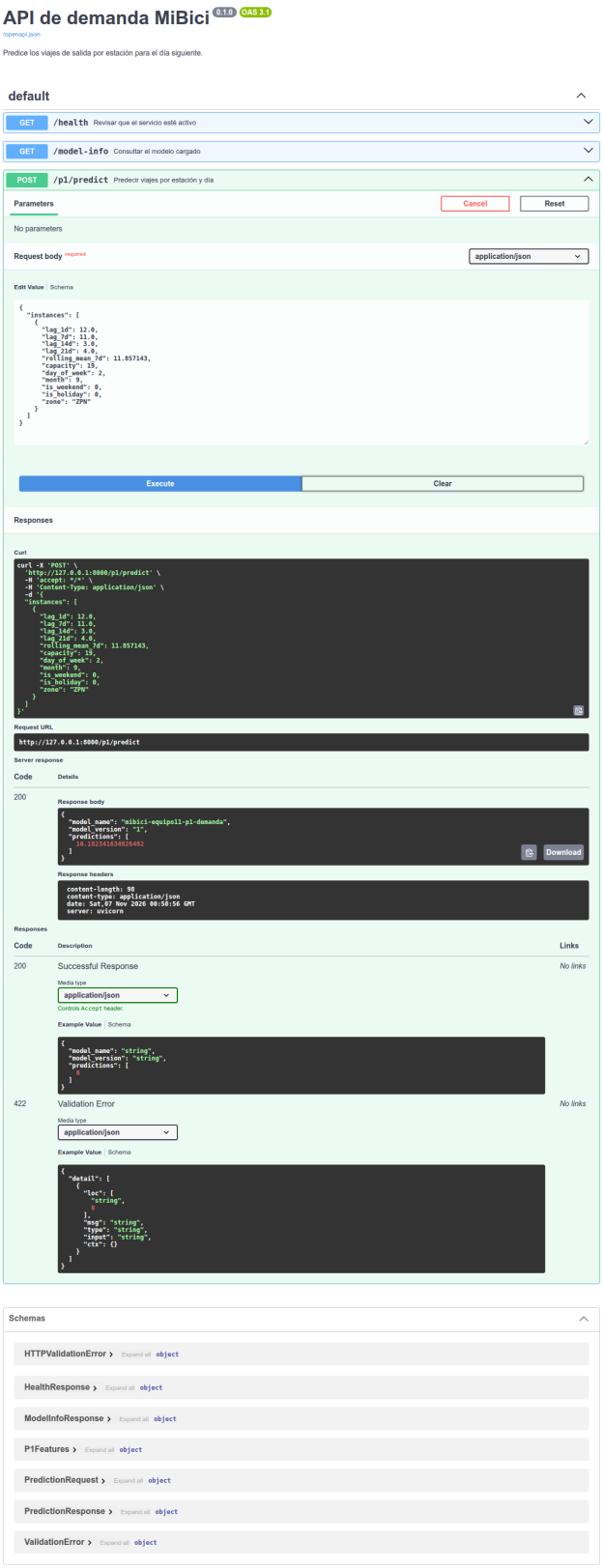
Contenedor en ejecución

```
$ docker run -d --name mibici-api -p 8000:8000 mibici-api
596ec7e2e943
$ docker ps
NAMES        IMAGE        STATUS                    PORTS
mibici-api   mibici-api   Up 10 seconds (healthy)   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
$ docker logs mibici-api
INFO:     Waiting for application startup.
2026-11-07 00:34:18.558 | INFO     | mi_bici.api.model:load_local_model:83 - Modelo local cargado: mibici-
equipo11-p1-demanda v1
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
imagen mibici-api 815MB
```

Peticiones a la API

```
$ curl http://127.0.0.1:8000/health
{"status":"ok"}
$ curl http://127.0.0.1:8000/model-info
{"model_name":"mibici-equipo11-p1-demanda","model_version":"1","run_id":"8b2ba4b2a00f4ccc9ec1422132f55c19"}
$ curl -X POST http://127.0.0.1:8000/p1/predict -H 'Content-Type: application/json' -d '{"instances":[{"lag_1d":12.0,"la
g_7d":11.0,"lag_14d":3.0,"lag_21d":4.0,"rolling_mean_7d":11.857143,"capacity":19,"day_of_week":2,"month":9,"is_weekend":
0,"is_holiday":0,"zone":"ZPN"}]}'
{"model_name":"mibici-equipo11-p1-demanda","model_version":"1","predictions":[10.182341634926482]}
HTTP 200
$ curl -X POST http://127.0.0.1:8000/p1/predict -d '{"instances":[{"month":13}]}'
{"detail":[{"loc":["body","instances",0,"lag_1d"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"lag_7d"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"lag_14d"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"lag_21d"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"rolling_mean_7d"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"capacity"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"day_of_week"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"month"],"msg":"Input should be less than or equal to
12","type":"less_than_equal"},{"loc":["body","instances",0,"is_weekend"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"is_holiday"],"msg":"Field
required","type":"missing"},{"loc":["body","instances",0,"zone"],"msg":"Field required","type":"missing"}]}
HTTP 422
```

API respondiendo desde el contenedor (Docker)



Extra: ONNX Runtime

Repo: https://github.com/oscartrix/TC5061-Proyecto-Equipo11

PR: #36 `feat/inferencia-onnx` (independiente del contenedor, #34)

Modelo: champion de P1 (`mibici-equipo11-p1-demanda` v1, HistGradientBoostingRegressor con `StandardScaler` y `OneHotEncoder`)

1. Qué se integró

• `scripts/export_onnx.py` convierte el pipeline de sklearn a `model.onnx` (skl2onnx 1.20, opset 17) y compara sus predicciones con las de sklearn.

• `mi_bici/api/onnx_model.py` expone la misma interfaz que el modelo actual (`name`, `version`, `run_id`, `predict`), por lo que `main.py` no cambió.

• `INFERENCE_BACKEND=onnx` activa ONNX Runtime; por defecto la API sigue con sklearn.

• OpenVINO no se integró: parte de un modelo ONNX, solo acelera en CPU/GPU Intel y para un árbol de decisión sobre pocas columnas no aporta más que ONNX Runtime.

2. Latencia (CPU local, 200 repeticiones, solo inferencia del modelo)

Filas	Motor	p50 ms	p95 ms	Filas/s
1	sklearn	5.09	6.88	196
1	ONNX Runtime	0.11	0.16	8 850
100	sklearn	5.27	6.44	18 977
100	ONNX Runtime	0.39	0.46	257 693
500	sklearn	7.56	11.67	66 116
500	ONNX Runtime	1.10	1.45	454 663

ONNX Runtime es entre 7 y 46 veces más rápido según el tamaño del lote. La mayor ganancia está en peticiones de una fila, que es el caso típico de la API.

3. Ventajas de cambiar

Ventaja	Detalle
Latencia	Menos de 1 ms por fila contra unos 5 ms de sklearn; el lote de 500 filas baja de 7.6 a 1.1 ms
Portabilidad	El `.onnx` no depende de la versión de sklearn ni de pickle; se puede cargar desde otros lenguajes
Seguridad	No se deserializa un pickle al arrancar; el `.onnx` es un grafo de operadores
Archivo más pequeño	`model.onnx` 450 KB contra `model.pkl` 725 KB
Dependencias	Con una API sin MLflow ni sklearn bastaría `onnxruntime` y `numpy`; hoy `model.py` aún importa MLflow, por lo que la imagen no se redujo en este PR

4. Costos y riesgos

Riesgo	Detalle
Diferencias numéricas	ONNX Runtime ejecuta los árboles en float32. En 20 000 filas de prueba reales, la diferencia media contra sklearn fue 0.088 viajes, p99 1.11 y máxima 16.9; 25.7 % de las filas cambian más de 0.01
Calidad global sin cambio	MAE contra los viajes reales: sklearn 5.698 y ONNX 5.693
Conversión por modelo	Cada familia (Ridge, RF, HGB) y cada reentrenamiento exige reconvertir y revisar la paridad
Parche necesario	skl2onnx 1.20 con onnx 1.23 rechaza booleanos en los árboles de HGB; `export_onnx.py` los convierte a enteros antes de serializar
Doble mantenimiento	Dos caminos de inferencia en la API y una dependencia opcional más (`requirements-onnx.txt`)
Doble precisión	El intento de usar `double` falló: ONNX Runtime no carga `TreeEnsembleRegressor` con salida double en esta versión

5. Recomendación

• Mantener sklearn como motor por defecto: la demanda por estación no tiene un requisito de latencia que 5 ms no cumpla.

• Usar ONNX para los modelos lineales (Ridge) o cuando haya un volumen alto de peticiones; ahí la paridad es exacta salvo redondeo.

• Antes de activarlo con el HGB, definir una tolerancia aceptada para los 25 % de filas que cambian, o reentrenar con características que no queden sobre los umbrales de los árboles (por ejemplo, sin valores enteros pegados al umbral).

• Si se adopta, refactorizar `model.py` para importar MLflow solo cuando se use y construir una segunda imagen con `onnxruntime`.

```
$ python scripts/export_onnx.py
diferencia media: 0.0880  p99: 1.1093  máxima: 16.8949
$ python scripts/benchmark_inference.py
filas | motor   | p50 ms | p95 ms | filas/s
    1 | sklearn |   5.09 |   6.88 | 196
    1 | onnx    |   0.11 |   0.16 | 8850
  100 | sklearn |   5.27 |   6.44 | 18977
  100 | onnx    |   0.39 |   0.46 | 257693
  500 | sklearn |   7.56 |  11.67 | 66116
  500 | onnx    |   1.10 |   1.45 | 454663
```